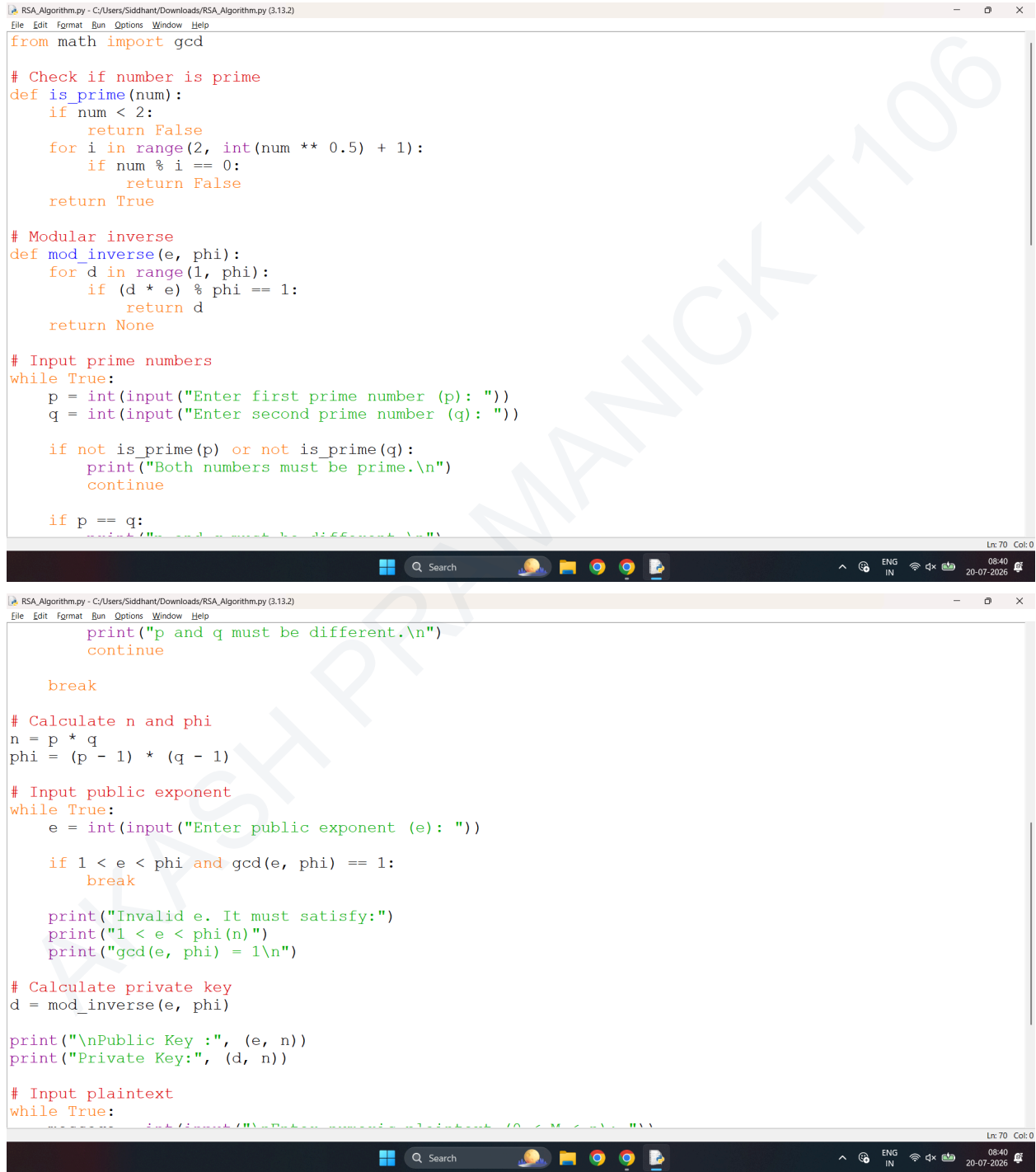


Aim: Implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations.

CLI

Input :



```

RSA_Algorithm.py - C:/Users/Siddhant/Downloads/RSA_Algorithm.py (3.13.2)
File Edit Format Run Options Window Help
from math import gcd

# Check if number is prime
def is_prime(num):
    if num < 2:
        return False
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            return False
    return True

# Modular inverse
def mod_inverse(e, phi):
    for d in range(1, phi):
        if (d * e) % phi == 1:
            return d
    return None

# Input prime numbers
while True:
    p = int(input("Enter first prime number (p): "))
    q = int(input("Enter second prime number (q): "))

    if not is_prime(p) or not is_prime(q):
        print("Both numbers must be prime.\n")
        continue

    if p == q:
        print("p and q must be different.\n")
        continue

    print("p and q must be different.\n")
    continue

    break

# Calculate n and phi
n = p * q
phi = (p - 1) * (q - 1)

# Input public exponent
while True:
    e = int(input("Enter public exponent (e): "))

    if 1 < e < phi and gcd(e, phi) == 1:
        break

    print("Invalid e. It must satisfy:")
    print("1 < e < phi(n)")
    print("gcd(e, phi) = 1\n")

# Calculate private key
d = mod_inverse(e, phi)

print("\nPublic Key :", (e, n))
print("Private Key:", (d, n))

# Input plaintext
while True:
    message = input("Enter message to encrypt (0 < M < n): ")
    if message:
        break
    else:
        continue
    
```

Name - Akash Pramanick

Roll No - T106

Subject - Cyber & Information Security

```

RSA_Algorithm.py - C:/Users/Siddhant/Downloads/RSA_Algorithm.py (3.13.2)
File Edit Format Run Options Window Help
    if e < phi and gcd(e, phi) == 1:
        break

    print("Invalid e. It must satisfy:")
    print("1 < e < phi(n)")
    print("gcd(e, phi) = 1\n")

# Calculate private key
d = mod_inverse(e, phi)

print("\nPublic Key :", (e, n))
print("Private Key:", (d, n))

# Input plaintext
while True:
    message = int(input("\nEnter numeric plaintext (0 < M < n): "))
    if 0 < message < n:
        break
    print("Message must be between 1 and", n - 1)

# Encryption
cipher = pow(message, e, n)

# Decryption
decrypted = pow(cipher, d, n)

print("\nEncrypted Message :", cipher)
print("Decrypted Message :", decrypted)

```

Output :

```

IDLE Shell 3.13.2
Python 3.13.2 (tags/v3.13.2:4f8bb39, Feb 4 2025, 15:23:48) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Siddhant/Downloads/RSA_Algorithm.py =====
Enter first prime number (p): 3
Enter second prime number (q): 11
Enter public exponent (e): 7

Public Key : (7, 33)
Private Key: (3, 33)

Enter numeric plaintext (0 < M < n): 5

Encrypted Message : 14
Decrypted Message : 5
>>>

```

GUI

Input :

Name - Akash Pramanick

Roll No - T106

Subject - Cyber & Information Security

```

RSA_Algorithm.py - C:/Users/Siddhant/Downloads/RSA_Algorithm.py (3.13.2)
File Edit Format Run Options Window Help

import tkinter as tk
from tkinter import messagebox
from math import gcd

# -----
# Prime Number Check
# -----
def is_prime(num):
    if num < 2:
        return False
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            return False
    return True

# -----
# Modular Inverse
# -----
def mod_inverse(e, phi):
    for d in range(1, phi):
        if (d * e) % phi == 1:
            return d
    return None

# -----
# RSA Function
# -----
def rsa_process():
    try:
        p = int(entry_p.get())
        q = int(entry_q.get())
        e = int(entry_e.get())
        message = int(entry_message.get())

        if not is_prime(p):
            messagebox.showerror("Error", "p must be a prime number.")
            return

        if not is_prime(q):
            messagebox.showerror("Error", "q must be a prime number.")
            return

        if p == q:
            messagebox.showerror("Error", "p and q must be different.")
            return

        n = p * q
        phi = (p - 1) * (q - 1)

        if not (1 < e < phi):
            messagebox.showerror("Error", "e must satisfy 1 < e < φ(n).")
            return

        if gcd(e, phi) != 1:
            messagebox.showerror("Error", "e must be coprime with φ(n).")
            return
    
```

Name - Akash Pramanick

Roll No - T106

Subject - Cyber & Information Security

```

RSA_Algorithm.py - C:/Users/Siddhant/Downloads/RSA_Algorithm.py (3.13.2)
File Edit Format Run Options Window Help

    cipher = pow(message, e, n)
    decrypted = pow(cipher, d, n)

    result.config(
        text=f"""PUBLIC KEY    : ({e}, {n})

PRIVATE KEY : ({d}, {n})

ENCRYPTED MESSAGE :
{cipher}

DECRYPTED MESSAGE :
{decrypted}"""
    )

    except ValueError:
        messagebox.showerror("Input Error", "Please enter valid integers.")

# -----
# Clear Function
# -----
def clear_fields():
    entry_p.delete(0, tk.END)
    entry_q.delete(0, tk.END)
    entry_e.delete(0, tk.END)
    entry_message.delete(0, tk.END)
    result.config(text="")

RSA_Algorithm.py - C:/Users/Siddhant/Downloads/RSA_Algorithm.py (3.13.2)
File Edit Format Run Options Window Help

# -----
# GUI
# -----
root = tk.Tk()
root.title("RSA Encryption & Decryption")
root.geometry("650x650")
root.configure(bg="#1E1E2F")
root.resizable(False, False)

# =====
# Title
# =====
title = tk.Label(
    root,
    text="RSA ENCRYPTION & DECRYPTION",
    bg="#1E1E2F",
    fg="#00D4FF",
    font=("Arial", 20, "bold")
)
title.pack(pady=20)

# =====
# Input Frame
# =====
frame = tk.Frame(root, bg="#2A2A40", bd=3, relief="ridge")
frame.pack(padx=25, pady=10, fill="x")

labels = [

```

Name - Akash Pramanick

Roll No - T106

Subject - Cyber & Information Security

```

RSA_Algorithm.py - C:/Users/Siddhant/Downloads/RSA_Algorithm.py (3.13.2)
File Edit Format Run Options Window Help
    "Prime Number (p)",
    "Prime Number (q)",
    "Public Exponent (e)",
    "Numeric Plaintext"
]

entries = []

for i, text in enumerate(labels):
    tk.Label(
        frame,
        text=text,
        bg="#2A2A40",
        fg="white",
        font=("Arial", 11, "bold")
    ).grid(row=i, column=0, padx=15, pady=12, sticky="w")

    entry = tk.Entry(
        frame,
        width=25,
        font=("Arial", 12),
        bd=2,
        relief="groove"
    )
    entry.grid(row=i, column=1, padx=15, pady=12)
    entries.append(entry)

entry_p, entry_q, entry_e, entry_message = entries

Ln: 247 Col: 0

# =====
# Buttons
# =====
button_frame = tk.Frame(root, bg="#1E1E2F")
button_frame.pack(pady=20)

encrypt_btn = tk.Button(
    button_frame,
    text="Encrypt & Decrypt",
    font=("Arial", 12, "bold"),
    bg="#00C853",
    fg="white",
    width=18,
    cursor="hand2",
    command=rsa_process
)
encrypt_btn.grid(row=0, column=0, padx=10)

clear_btn = tk.Button(
    button_frame,
    text="Clear",
    font=("Arial", 12, "bold"),
    bg="#FF5252",
    fg="white",
    width=12,
    cursor="hand2",
    command=clear_fields
)

```

Name - Akash Pramanick

Roll No - T106

Subject - Cyber & Information Security

```

RSA_Algorithm.py - C:/Users/Siddhant/Downloads/RSA_Algorithm.py (3.13.2)
File Edit Format Run Options Window Help

# =====
# Result Frame
# =====
result_frame = tk.LabelFrame(
    root,
    text="Result",
    bg="#2A2A40",
    fg="#FFD54F",
    font=("Arial", 13, "bold"),
    padx=15,
    pady=15
)
result_frame.pack(fill="both", padx=20, pady=15, expand=True)

result = tk.Label(
    result_frame,
    text="",
    bg="#2A2A40",
    fg="FFFFFF",
    justify="left",
    anchor="nw",
    font=("Consolas", 12)
)

result.pack(fill="both", expand=True)

# =====
# Footer
# =====

result_frame.pack(fill="both", padx=20, pady=15, expand=True)

result = tk.Label(
    result_frame,
    text="",
    bg="#2A2A40",
    fg="FFFFFF",
    justify="left",
    anchor="nw",
    font=("Consolas", 12)
)

result.pack(fill="both", expand=True)

# =====
# Footer
# =====
footer = tk.Label(
    root,
    text="RSA Algorithm Simulator | TYCS Practical",
    bg="#1E1E2F",
    fg="gray",
    font=("Arial", 10)
)
footer.pack(side="bottom", pady=10)

root.mainloop()

```

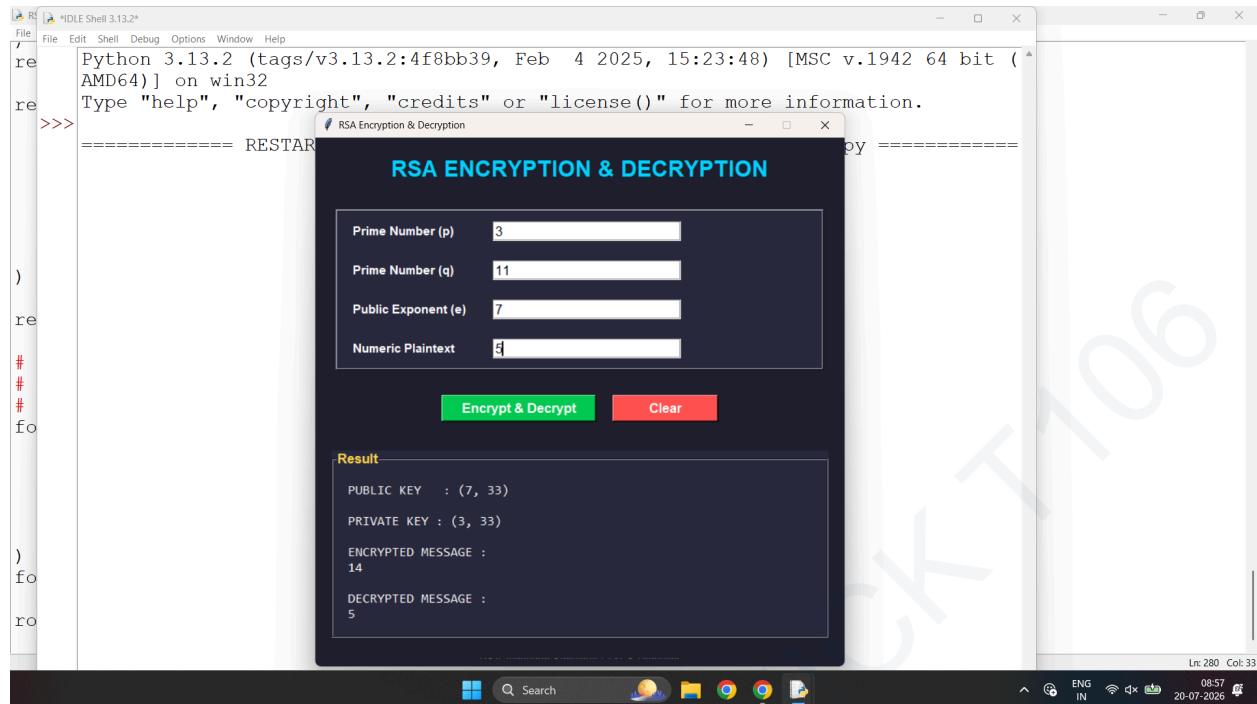
Output :

Name - Akash Pramanick

Roll No - T106

Subject - Cyber & Information Security

SHETH L.U.J. & SIR M.V. COLLEGE OF SCIENCE



Name - Akash Pramanick

Roll No - T106

Subject - Cyber & Information Security